

TechSapo 使い方ガイド

Claude Code 起動から Wall-Bounce 実行まで

TechSapo Documentation Team

2025-10-07

Contents

1	Claude Code での壁打ち Wall-Bounce の使い方	2
1.1	基本原則	2
1.2	基本的な使い方	3
1.3	タスク別の自動ルーティング	5
1.4	実際の壁打ちフロー例	6
1.5	壁打ちのルール	8
1.6	品質保証の仕組み	9
1.7	GPT-5 vs GPT-5 Codex の使い分け	10
1.8	よくある質問	12
1.9	詳細情報の参照先	13
1.10	まとめ	14

1 Claude Code での壁打ち Wall-Bounce の使い方

1.1 基本原則

Claude Code を使う際の絶対的な原則

物事の解決には 2 以上の LLM モデルとの壁打ち必須

すべての問い合わせで、Claude Code が司令塔となり、複数の LLM へクエリを発行して回答を統合します。

1.2 基本的な使い方

1.2.1 1. Claude Code に問い合わせる

あなた：「TypeScriptでバグを修正してください」

Claude Code があなたの問い合わせを受け付けます。

1.2.2 2. Claude Code の内部処理 自動

Claude Code は以下を自動で実行します

ステップ 1: タスク分析

- キーワード検出: "TypeScript", "バグ修正"
- ドメイン判定: "coding"
- タスクタイプ決定: "coding"

ステップ 2: プロバイダー選択

コーディングタスク検出

↓

Tier 2 Coding特化モデルを自動選択:

- GPT-5 Codex OpenAI
- Qwen3 Coder OpenRouter
- Gemini 2.5 Flash 検証用・異なるベンダー

ステップ 3: 壁打ち実行 並列

3つのLLMへ同時にクエリ発行

↓

各LLMが独立して回答

↓

回答の類似度・品質を評価

↓

合意度 Consensus が0.6以上か

YES → 次のステップへ

NO → 追加LLMを実行 最大6ラウンド

ステップ 4: 統合

Claude Sonnet 4.5 アグリゲーター が

- 3つの回答を分析
- 最良の部分を抽出
- 一貫性のある統合回答を生成

ステップ 5: あなたへ回答

Claude Code: 「バグを修正しました。以下のコードをご確認ください...」

1.3 タスク別の自動ルーティング

1.3.1 コーディングタスク

あなたの問い合わせ例: - 「TypeScript で OAuth2 を実装して」 - 「このバグを修正して」 - 「コードレビューしてください」

自動選択される **LLM**: 1. GPT-5 Codex Primary 2. Qwen3 Coder Primary 3. Gemini 2.5 Flash Validation 4. Claude Sonnet 4.5 Aggregator

特徴: - 高速なコード生成 - TypeScript/JavaScript 最適化 - ミニマルな推論プロセス

1.3.2 分析タスク

あなたの問い合わせ例: - 「このシステムのセキュリティを分析して」 - 「アーキテクチャのトレードオフを評価して」 - 「パフォーマンス問題を調査して」

自動選択される **LLM**: 1. GPT-5 Standard Primary 2. Gemini 2.5 Pro Primary 3. Gemini 2.5 Flash Validation 4. Claude Sonnet 4.5 Aggregator

特徴: - 詳細な多段階推論 - 多角的な視点 - 構造化された分析

1.3.3 複雑タスク

あなたの問い合わせ例: - 「マイクロサービス移行の計画を立てて」 - 「この数学的証明を解いて」 - 「システム全体のリファクタリング戦略を提案して」

自動選択される **LLM**: 1. Gemini 2.5 Pro (Deep Think) Primary 2. GPT-5 Standard Primary 3. Claude Sonnet 4.5 Primary 4. Claude Opus 4.1 Aggregator

特徴: - 深い推論モード - 並列仮説検証 - 最も強力な統合エンジン

1.4 実際の壁打ちフロー例

1.4.1 例 1: コーディングタスク

あなた: 「TypeScriptでメールバリデーターを書いて」

Round 1: 並列実行

GPT-5 Codex → 「RFC5322準拠のバリデーター実装」

Qwen3 Coder → 「詳細なエラーメッセージ付き実装」

Gemini Flash → 「シンプルで高速な実装」

品質チェック

Consensus: 0.78 合格

Confidence: 0.85 高品質

→ 3つの実装が類似しており、品質も高い

統合 Claude Sonnet 4.5

3つの実装の良い部分を統合:

- GPT-5 CodexのRFC5322準拠
- Qwen3 Coderのエラーメッセージ
- Gemini Flashのシンプルさ

→ 最適な実装を生成

Claude Code → あなた

「TypeScriptのメールバリデーターを実装しました。
RFC5322に準拠し、詳細なエラーメッセージ付きです…」

1.4.2 例 2: 分析タスク

あなた: 「平文パスワード保存のセキュリティリスクを分析して」

Round 1: 並列実行

GPT-5 → 詳細な多段階分析

"Structuring security analysis"

"Analyzing security implications"

"Summarizing password security"

Gemini 2.5 → 高品質な分析
Pro

- Core Risks
- Attack Scenarios
- Compliance Issues

Gemini 2.5 → 高速な要点整理
Flash

品質チェック

Consensus: 0.72 合格
Confidence: 0.88 高品質
→ 分析内容が一致しており、信頼性が高い

統合 Claude Sonnet 4.5

3つの分析を統合:

- 最も包括的な脅威モデル
- 具体的な攻撃シナリオ
- 実践的な対策案

→ 統合されたセキュリティ分析

Claude Code → あなた

「平文パスワード保存のセキュリティリスクを分析しました。
主なリスクは...」

1.5 壁打ちのルール

1.5.1 最低ラウンド数: 3 回

Claude Code は必ず最低 3 回の壁打ちを実施します

1. **Round 1:** 初回 LLM 群 通常 2 つ
2. **Round 2:** 異なるベンダーの LLM で検証
3. **Round 3:** さらに異なるベンダーで検証

1.5.2 最大ラウンド数: 6 回

品質が基準に達しない場合、最大 6 回まで追加の LLM を実行します。

1.5.3 ベンダー切り替えルール

必須: 前回と異なるベンダーの LLM を選択

例:

Round 1: OpenAI GPT-5 Codex

Round 2: Google Gemini 2.5 Pro ← 異なるベンダー

Round 3: Anthropic Sonnet 4.5 ← 異なるベンダー

理由: 異なるベンダーの LLM を使うことで、バイアスを減らし、多様な視点を得る

1.6 品質保証の仕組み

1.6.1 コンセンサス Consensus

複数の LLM 回答がどれだけ一致しているかを示す指標。

計算方法:

各 ペア の 類 似 度 を 計 算

↓

平 均 を 取 る

↓

0.0～1.0 の ス コ ア

判定基準: - 0.6 以上: 合格 回答が一致している - 0.6 未満: 不合格 追加ラウンド実行

1.6.2 信頼度 Confidence

各 LLM 回答の品質を示す指標。

評価項目: - 具体性 - 構造化 - 専門用語の適切さ - 論理的整合性

判定基準: - 0.7 以上: 高品質 - 0.5～0.7: 要検証 - 0.5 未満: 再生成

1.6.3 Early Stopping

品質基準を満たした時点で即座に終了し、無駄な LLM 呼び出しを回避。

条件:

Consensus $\geq$ 0.7

AND

Confidence $\geq$ 0.7

AND

実行 LLM 数 $\geq$ 必要数

↓

終了して統合ステップへ

1.7 GPT-5 vs GPT-5 Codex の使い分け

Claude Code は自動で最適なモデルを選択しますが、違いを理解しておくと有用です。

1.7.1 GPT-5 標準

使われるタスク: - 分析タスク - 文章生成 - 複雑な推論

特徴: - 詳細な多段階推論 - 「Structuring → Analyzing → Summarizing」の3段階思考 - 多角的な視点

推論プロセス例:

Thinking 1: "Structuring security analysis"

→ 分析の枠組みを構築中

Thinking 2: "Analyzing security implications"

→ セキュリティへの影響を分析中

Thinking 3: "Summarizing password security"

→ パスワードセキュリティをまとめ中

Output: 詳細なセキュリティ分析レポート

1.7.2 GPT-5 Codex

使われるタスク: - コーディング - デバッグ - コードレビュー

特徴: - ミニマルな推論 「Less is more」 - 適応推論 Adaptive Reasoning - 即座に高品質コード生成

推論プロセス例:

Thinking: "Drafting TypeScript email validator response"

→ バリデーター実装をドラフト中

Output: 即座に完成度の高いコード

1.7.3 自動選択の実例

あなたが「TypeScript でバグ修正」と依頼

Claude Code判定: taskType = "coding"

↓

自動選択: GPT-5 Codex

↓

特徴： ミニマル推論、即座にコード生成

あなたが「セキュリティ分析」と依頼

Claude Code判定: taskType = "analysis"

↓

自動選択: GPT-5 Standard

↓

特徴： 詳細推論、多角的分析

1.8 よくある質問

1.8.1 Q1: 壁打ちは毎回必須ですか

A: はい。TechSapo の絶対的原則として、すべてのクエリで最低 2 つの LLM による壁打ちを実施します。

1.8.2 Q2: どの LLM が選ばれるか指定できますか

A: いいえ。Claude Code がタスクタイプを自動判定し、最適な LLM を選択します。これにより、常に最高品質の回答を保証します。

1.8.3 Q3: 壁打ちに時間がかかりませんか

A: 並列実行により、複数 LLM を同時に呼び出すため、単一 LLM の 2 倍程度の時間で完了します。品質向上のトレードオフとして許容範囲です。

1.8.4 Q4: コンセンサスが低い場合はどうなりますか

A: Claude Code が自動で追加ラウンドを実行し、より多くの LLM から意見を集めます 最大 6 ラウンド。

1.8.5 Q5: 途中経過は見られますか

A: はい。Server-Sent Events SSE モードを使うと、各 LLM の実行状況をリアルタイムで確認できます。

1.9 詳細情報の参照先

より詳しい情報が必要な場合は、以下のドキュメントを参照してください

1.9.1 システム詳細

- [/ai/prj/techdev/docs/SYSTEM_FLOW_EXPLANATION.md](#) - 詳細なシステムフロー
- [/ai/prj/techdev/docs/WALL_BOUNCE_SYSTEM.md](#) - Wall-Bounce の技術詳細

1.9.2 アーキテクチャ

- [/ai/prj/techdev/docs/ARCHITECTURE_OVERVIEW.md](#) - システムアーキテクチャ
- [/ai/prj/techdev/docs/LLM_PROVIDERS_GUIDE.md](#) - LLM プロバイダーガイド

1.9.3 GPT-5/Codex

- [/ai/prj/techdev/docs/GPT5_CODEX_FINAL_SUMMARY.md](#) - 検証結果サマリー
- [/ai/prj/techdev/docs/GPT5_VS_GPT5_CODEX_実証結果.md](#) - 使い分け実証

1.9.4 トラブルシューティング

- [/ai/prj/techdev/docs/TROUBLESHOOTING.md](#) - 問題解決ガイド

1.9.5 開発者向け

- [/ai/prj/techdev/CLAUDE.md](#) - Claude Code 開発ガイド
- [/ai/prj/techdev/docs/API_REFERENCE.md](#) - API 仕様

1.10 まとめ

1.10.1 TechSapo Wall-Bounce の特徴

1. 自動: Claude Code がすべて自動処理
2. 高品質: 複数 LLM の協調で高精度回答
3. 信頼性: コンセンサス・品質チェック
4. 透明性: 判断プロセスが明確

1.10.2 あなたがすること

1. Claude Codeに問 い 合 わ せ る
2. Claude Codeが自動で壁打ち実行
3. 統合された高品質回答を受け取る

シンプルです。

1.10.3 重要な原則

物事の解決には 2 以上の LLM モデルとの壁打ち必須

この原則により、TechSapo は常に高品質で信頼性の高い回答を提供します。

作成: TechSapo Documentation Team 更新: 2025-10-07 バージョン: 3.0